

Skills Recommender System

Information Retrieval

481SM

05/02/2026



Skills Recommender System

Implementation in an HR commercial software

The Software: Enterprise OS by KS-Agents



ProductsSolutionsHow it WorksPlansBlog

 Sign In

AI-POWERED ORGANIZATIONAL INTELLIGENCE

Reveal. Activate. Transform.

Turn workforce data into strategic assets. We analyze turnover, productivity, and team dynamics to replace intuition with clear, actionable evidence—instantly.

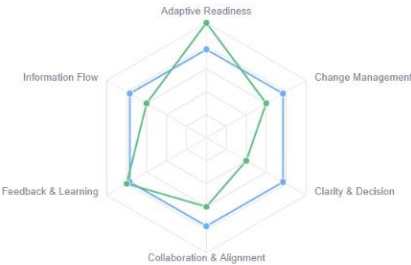
Book a ConsultationGet Started Now!

Perception vs Reality

SFI from Health Screener vs AFS from Agility Check

● Perception (SFI)

● Reality (AFS)



● Perception (SFI)

System Flow Index **76.9%**

Healthy

Perception snapshot from the Organizational Health Screener.

● Reality Check (AFS)

Adaptive Readiness	100.0
Change Management	60.0
Clarity & Decision Architecture	40.0
Collaboration & Alignment	60.0
Feedback & Learning	80.0
Information Flow	60.0

Luca Ricatti

SM3800083

Skills Recommender System

05/02/2026

3/22

Call for Startups: OfficineEdison

Cerchiamo soluzioni tecnologiche e servizi innovativi che possano:

- ✓ Mappare in modo certificato e aggiornabile le competenze tecniche e trasversali presenti in azienda;
- ✓ Individuare gap e trend futuri in relazione all'evoluzione dei mestieri e dei business aziendali;
- ✓ Favorire la mobilità interna e il matching tra competenze, ruoli e percorsi di sviluppo;
- ✓ Supportare il reskilling e l'upskilling in logica personalizzata e adattiva;

Databases

Occupational Information Databases

- ESCO: European Skills, Competences, Qualifications and Occupations
 - <https://esco.ec.europa.eu/en/use-esco/download>
- O*NET: USA source of occupational information
 - <https://www.onetcenter.org/database.html#all-files>

Database	Occupations	Items	Items #	Relations	Avg rels	Sparsity	Feedback
Esco	3.039	Skills	13.939	126.051	41,5	0,30%	Binary
O*Net	1.016	Technology skills	8.786	32.773	32,3	0,37%	Weighted
O*Net	1.016	Tasks	18.796	17.951	17,7	0,09%	Weighted

Databases: Esco

Skills

Occupations		Algorithms	Build recommender systems	Data mining	Data models	Perform risk analysis	Query languages
	...						
	Artificial Intelligence engineer	1	1	0	1	0	0
	...						
	Energy systems engineer	0	0	0	0	1	0
	...						
	Data Engineer	0	0	0	1	0	0
	Data Scientist	0	1	1	1	0	1
	...						

Databases: O*NET

Technology Skills

	GIT	Oracle Taleo	HPL	C#	...
...					
Business Intelligence Analysts	1	0.5	0.5	1	
...					

Tasks

	Maintain library of model documents, templates, or other reusable knowledge assets.	Install or configure Web server software or hardware to ensure that directory structure is well-defined, logical, and secure, and that files are named properly.	Set up or maintain monitoring tools on Web servers or Web sites.	...
...				
Business Intelligence Analysts	3.41	4.64	4.19	
...				

Occupations

From Static Taxonomies to Generative Profiling

- **Static vs. Dynamic**

- Official databases (ESCO, O*NET) are snapshots of the present. They describe established roles.

- **The Market Gap**

- Modern organizations require **hybrid roles** that do not exist yet in the database.

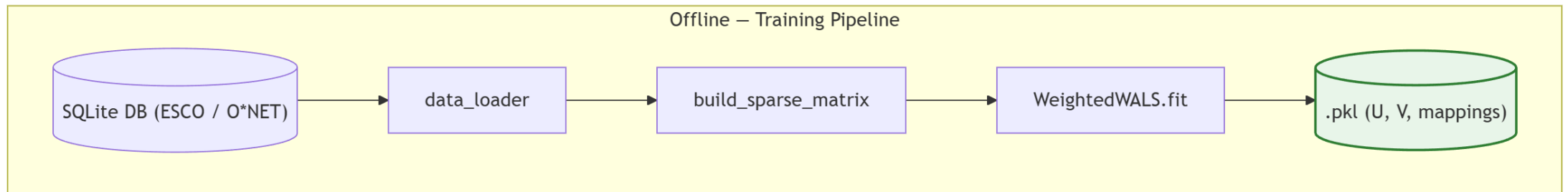
- **Our Objective**

- Allow users to define a "Target Profile" by combining seed skills from distinct domains.
- Use the latent space of the database to infer the missing "bridge skills" necessary to make that professional profile complete and realistic.

End-to-end pipeline

Offline (training):

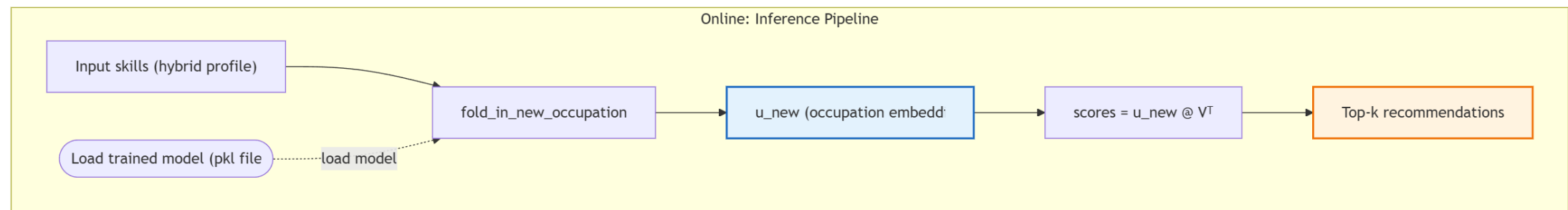
- Data is loaded from SQLite (ESCO or O*NET): occupations, skills (or tasks / technology examples), and their relations. A sparse Occupation $\times$ Skill matrix is built (binary for ESCO, weighted for O*NET). Weighted WALS learns latent factors U (occupation embeddings) and V (skill embeddings). The model plus index mappings and hyperparameters are saved as a `.pkl` file. This step is run once or periodically and does not depend on user input.



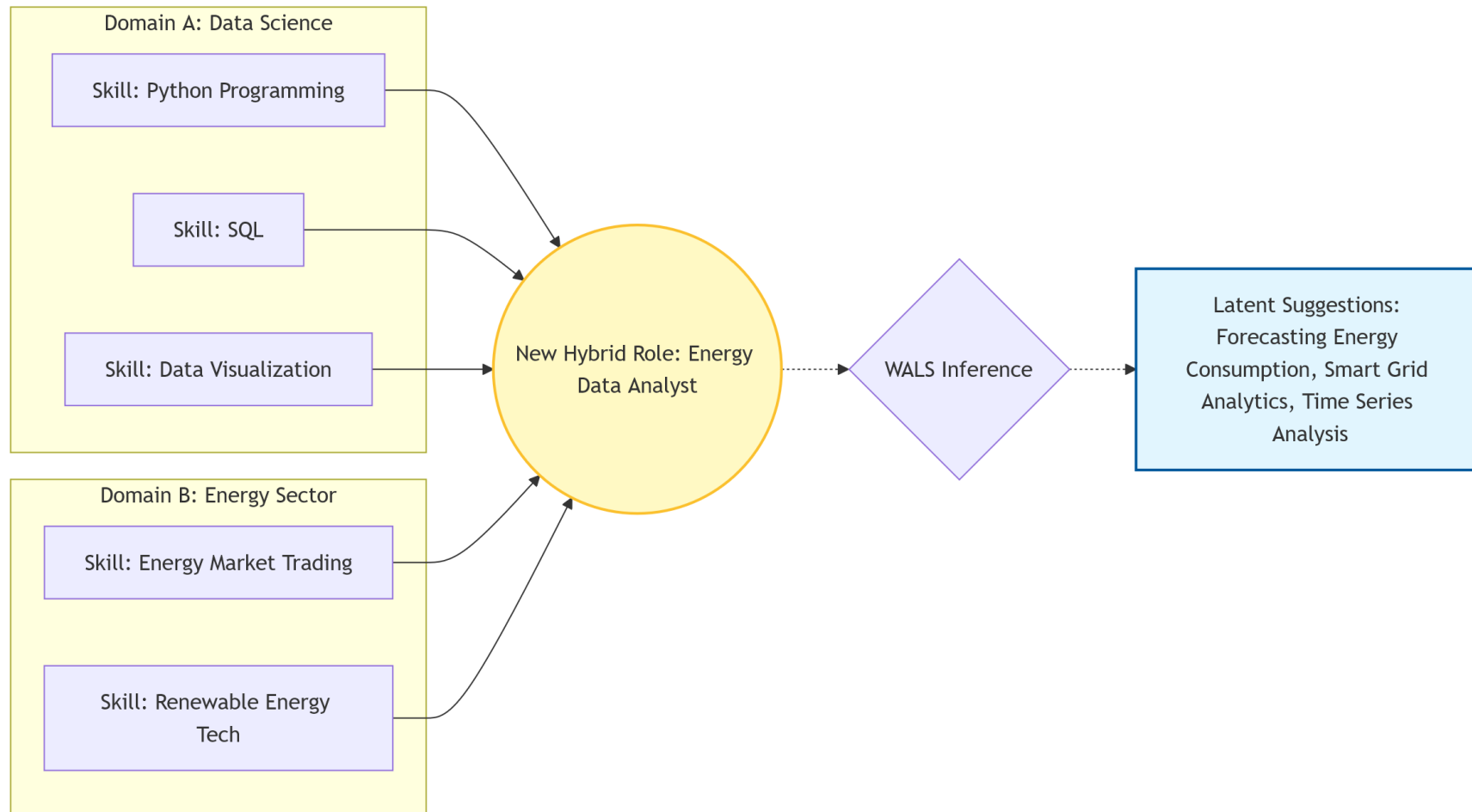
End-to-end pipeline

Online (inference):

- When the user defines a new or hybrid profile by selecting a set of skills, we **fold in** that profile: solve the WALS linear system (with V fixed) to obtain the occupation embedding u_{new} . We then score all skills as $u_{new}^T V$, sort by score, and return the top-k recommendations. No retraining is needed; the saved V and mappings are reused.



From Static Taxonomies to Generative Profiling



Building the Occupation $\times$ Skill matrix

The Data Nature:

- unlike movie ratings where missing data implies "uncertainty" (user hasn't seen the movie), in ESCO/O*NET missing relations often imply **rejection** (a Data Analyst *definitely* doesn't need "Perform sensory evaluation of food products").

The Sparsity Problem:

- The matrix is $\sim 99.7\%$ sparse.
- Treating all zeros as "Unobserved" ($w = 0$) $\rightarrow$ Model learns nothing about what implies *not* having a skill.
- Treating all zeros as "Negative" ($w = 1$) $\rightarrow$ Model is overwhelmed by zeros; predicts 0 everywhere.

The Challenge of "Hard Zeros"

- **The Solution: Weighted Confidence (w_{ui})**

- We treat "1s" (Observed) as **Preferences** with high confidence ($w_{obs} = 1.0$).
- We treat "0s" (Unobserved) as **Weak Negatives** with low confidence ($w_0 \approx 0.01$).
- **Impact:** This prevents the massive amount of irrelevant skills from crushing the latent signal of the relevant ones.

Formula

- Minimize the *Weighted Squared Error*:

$$\min_{u,v} \sum_{u,i} w_{ui} (r_{ui} - u_i^T v_i)^2 + \lambda (\|u\|^2 + \|v\|^2)$$

Building the Occupation × Skill matrix

Turn (occupation, skill [, weight]) into a CSR, (Compressed Sparse Row) sparse matrix for WALs.

- **Binary** (ESCO): every relation gets weight `1.0`.
- **Weighted** (O*NET): use importance/derived weight as third element.

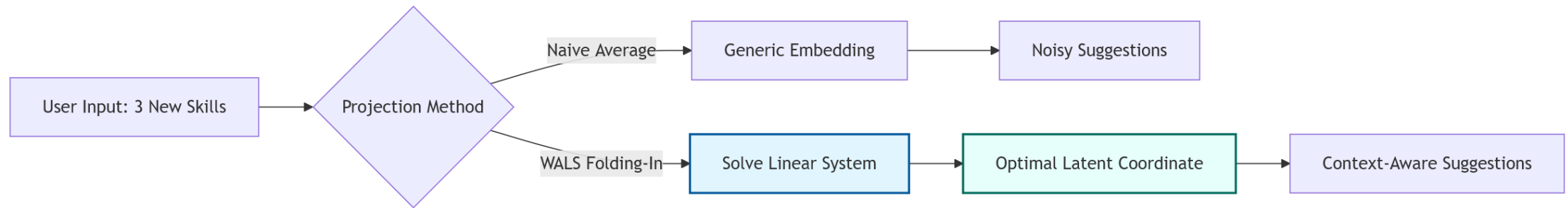
Result: Matrix $M \times N$ with only non-zero entries stored; supports row/column iteration in WALs.

```
# trainer.py - build_sparse_matrix
rows, cols, data = [], [], []
for rel in occupation_skill_rels:
    occ_idx = occupation_to_idx[occ_uri]
    skill_idx = skill_to_idx[skill_uri]
    rows.append(occ_idx); cols.append(skill_idx); data.append(weight)
M = len(occupation_to_idx)
N = len(skill_to_idx)
matrix = csr_matrix((data, (rows, cols)), shape=(M, N))
```

Advanced Inference: Implementing WALS Folding-In

- **The Use Case**
 - Real-time generation of "Hybrid Profiles" that do not exist in the training set.
- **Approach A (Naive - Deprecated): Centroid Averaging.**
 - Taking the average of input skill vectors.
 - *Flaw:* Ignores latent space density and the negative information (what the job isn't).
- **Approach B (Implemented - *Folding-In*): Linear System Solution.**
 - Fixed the trained Item Matrix ($\vec{V}$) and solve for the new User Vector (u_{new}) using the WALS update rule.
 - It mathematically finds the optimal projection into the latent space that respects both the selected skills and the weighted unselected ones.
- **Result**
 - The model suggests skills based on latent structure rather than simple vector proximity.

Advanced Inference: Implementing WALS Folding-In



- **Result**

- The model suggests skills based on latent structure rather than simple vector proximity.

Advanced Inference: Implementing WALS Folding-In

- User defines a new “hybrid” role by selecting a set of skills; the occupation is not in the training set.
- Keep trained V fixed; solve the same linear system as in training for a single “user” whose observed entries are the selected skills (with optional weights).
- **Result:** u_{new} = optimal occupation vector in the learned latent space; respects both selected skills and the w_0 penalty for non-selected ones.

```
# recommender.py – fold_in_new_occupation
V_all_sum = model.item_factors.T @ model.item_factors
A_obs = sum over input skills: w_ij * v_j v_j^T
b_obs = sum over input skills: w_ij * 1.0 * v_j
A_nobs = w_0 * (V_all_sum - A_obs)
A = A_obs + A_nobs + regularization * I
u_new = solve(A, b_obs)
```

Scoring and ranking skills

- **After folding-in:** We have u_{new} (occupation embedding).
- **Scores:** For every skill j , $score_j = u_{new}^T v_j$ (dot product in latent space).
- **Rank:** Sort skills by score descending; optionally remove input skills; return top-k.
- **Contrast:**
 - **Simple average:** $u_{position} = (1/|S|) \sum v_j$ over input skills. Fast, but ignores w_0 and regularization;
 - **Folding-in:** Solve $(A_{obs} + A_{nobs} + \lambda I) u_{new} = b_{obs}$ with V fixed.

```
# recommender.py - recommend_skills (after fold-in)
position_embedding = fold_in_new_occupation(model_data, input_skill_uris, ...)
scores = position_embedding @ model.item_factors.T
scored_skills = [(idx_to_skill[j], score) for j, score in enumerate(scores)
                  if skill not in input_set]
recommendations = sorted(scored_skills, key=score, reverse=True)[:top_k]
```

Main Hyperparameters: tuning the model

Parameter	Role	Typical Range
Factors (k)	Latent dimension; higher = more capacity	20 - 100
Regularization (λ)	Prevents overfitting	0.01 - 0.5
w_0	Weight for “non-relevant” (missing) entries	0.01 (soft) 0.1 - 1.0 (stronger penalty)
Iterations	WALS rounds; more = better convergence	10 - 20

Skills suggestion for Hybrid Role

Skills

[+ Add Skill](#)

Knowledge

Cloud technologies	Proficiency: 3/5		
Computer science	Optional Proficiency: 3/5		
Statistics	Optional Proficiency: 3/5		
Unstructured data	Optional Proficiency: 3/5		

Skills

Use data processing techniques	Proficiency: 3/5		
Develop data processing applications	Optional Proficiency: 3/5		
Manage quantitative data	Optional Proficiency: 3/5		
Use databases	Optional Proficiency: 3/5		

Suggested Skills

AI-powered skill recommendations based on your current skills

Knowledge

Score: 0.2183

+

Business intelligence

The tools used to transform large amounts of raw data into relevant and helpful business information...

Score: 0.2063

+

Visual presentation techniques

The visual representation and interaction techniques, such as histograms, scatter plots, surface plo...

Score: 0.2012

+

Information structure

The type of infrastructure which defines the format of data: semi-structured, unstructured and struc...

Skills

Score: 0.1858

+

Execute analytical mathematical calculations

Apply mathematical methods and make use of calculation technologies in order to perform analyses and...

Score: 0.1855

+

Perform data mining

Explore large datasets to reveal patterns using statistics, database systems or artificial intelligence...

Score: 0.1828

+

Perform project management

Manage and plan various resources, such as human resources, budget, deadline, results, and quality n...

Future Work

GPU Acceleration

Move WALS factor updates to GPU (e.g. PyTorch, CuPy). Matrix operations ($V^T V$, linear solves) scale with $M \times N \times k$ and are highly parallelisable; GPU can cut training time significantly on large datasets.

Re-Ranking

Add a second-stage re-ranker on top of WALS scores. Use a lightweight model (e.g. pairwise or listwise learning-to-rank) to incorporate extra signals (skill popularity, diversity, co-occurrence) and refine the final top-k list.

References

Theoretical Foundations & Algorithms

- **Hu, Y., Koren, Y., & Volinsky, C. (2008).** *Collaborative Filtering for Implicit Feedback Datasets*. Proceedings of the IEEE International Conference on Data Mining (ICDM).
 - Note: This is the foundational paper for the Weighted Matrix Factorization (WMF) logic used in our project.
- **Nenzi, L. (2024).** *Information Retrieval - Lecture 9: Recommender Systems and Matrix Factorisation*. University of Trieste.
 - Note: This lecture provided the theoretical framework for Latent Factor Models and the WALs optimization process.

Software Libraries & Tools

- **Frederickson, B. (2023).** *Implicit: Fast Python Collaborative Filtering for Implicit Datasets* (Version 0.7) [Software]. Available from <https://github.com/benfred/implicit>.
 - Note: This package served as the architectural benchmark for our implementation. We utilized its structural design as inspiration for our custom WALs classes and the `MockImplicitModel` wrapper.
- **Harris, C. R., et al. (2020).** *Array programming with NumPy*. Nature, 585(7825), 357-362.
- **Virtanen, P., et al. (2020).** *SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python*. Nature Methods, 17(3), 261-272.

Data Sources

- **European Commission.** *ESCO (European Skills, Competences, Qualifications and Occupations)*. <https://ec.europa.eu/esco/>.
- **U.S. Department of Labor.** *ONET OnLine Database**. <https://www.onetonline.org/>.

Project & Personal Repositories

- **Skill-Recommender-System** [Source Code]. GitHub. <https://github.com/egesia1/Skill-Recommendation-System>.
 - Note: This repository contains the complete implementation of the recommendation engine, including data processing and WALs optimization.